



Floci.io - 51 AWS Services Complete Hindi Guide for .NET Developers

Teacher Mode: Friendly | Language: Simple Hinglish/Hindi |
Audience: .NET Developer learning Cloud

Goal: Local machine pe `localhost:4566` par AWS seekhna — bina real account ke, bina bill ke!



Introduction: Floci.io Kya Hai?

Floci.io ek **local AWS emulator** hai jo aapke laptop pe ek **mini AWS cloud** banata hai using Docker.

Address: `http://localhost:4566`

Real-Life Example: > Real AWS = Highway pe ₹500/litre wala petrol daal ke gaddi chalana (expensive, risky for beginners) > > Floci = Driving school ka simulator — galti karo, seekho, practice karo — **koi bill nahi!** 🚗💡

.NET Developer ke liye kyun useful? - Visual Studio mein `F5` dabao → breakpoint laga ke debug karo - S3, DynamoDB, Lambda — sab local mein, instant response - CI/CD pipelines test karo bina real AWS account ke - Code same rahta hai — sirf endpoint URL change hoti hai (`localhost:4566` → real AWS)



Dummy Credentials (Floci ke liye)

```
Access Key ID:    test
Secret Access Key: test
Region:          us-east-1
Account ID:      000000000000
```

Ye **fake/dummy** credentials hain — Floci real AWS ko call nahi karta, isliye ye chal jaate hain!

51 Services Explained

◆ 1. S3 — Simple Storage Service

Kya hai? Virtual dabba (bucket) jismein aap files store kar sakte ho — images, PDFs, videos, backups.

Real-Life Example: School ka **almari** — har class ka alag shelf. S3 bucket = shelf, file = book.

AWS Equivalent: AWS S3

Floci Local Usage:

```
aws --endpoint-url=http://localhost:4566 s3 mb s3://meri-bucket
aws --endpoint-url=http://localhost:4566 s3 cp photo.jpg s3://meri-bucket/
```

.NET Developer:

```
var s3 = new AmazonS3Client(
    new BasicAWSCredentials("test","test"),
    new AmazonS3Config { ServiceURL="http://localhost:4566", ForcePathStyle=true });
await s3.PutObjectAsync(new PutObjectRequest { BucketName="meri-bucket",
    Key="cow.jpg", FilePath="cow.jpg" });
```

Project Example: Cow ki photo upload karo, S3 mein store karo, baad mein download karo.

◆ 2. SQS — Simple Queue Service

Kya hai? Message ki **line/queue**. Kaam ko line mein lagao, system overload nahi hoga.

Real-Life Example: Bank ka **token system** — 20 log aaye, sabko token mila, ek-ek counter pe bulaya.

AWS Equivalent: AWS SQS

Floci Local Usage:

```
aws --endpoint-url=http://localhost:4566 sqs create-queue --queue-name cow-jobs
aws --endpoint-url=http://localhost:4566 sqs send-message --queue-url http://
localhost:4566/000000000000/cow-jobs --message-body "ProcessCow001"
```

.NET Developer:

```
var sqs = new AmazonSQSClient(creds, new AmazonSQSConfig { ServiceURL="http://localhost:4566" });
await sqs.SendMessageAsync("http://localhost:4566/000000000000/cow-jobs",
    "ProcessCow001");
```

Project Example: 1000 cows ka data process karna — queue mein dalo, worker ek-ek kar ke process kare.

◆ 3. SNS — Simple Notification Service

Kya hai? Pub-Sub messaging. Ek message bhejo, bahut subscribers ko milega (SMS, Email, Lambda).

Real-Life Example: School ka **loudspeaker** — principal ne announcement ki, pure school mein sunayi di.

AWS Equivalent: AWS SNS

Floci Local Usage:

```
aws --endpoint-url=http://localhost:4566 sns create-topic --name cow-alerts
aws --endpoint-url=http://localhost:4566 sns subscribe --topic-arn arn:aws:sns:us-east-1:000000000000:cow-alerts --protocol email --notification-endpoint farmer@test.com
aws --endpoint-url=http://localhost:4566 sns publish --topic-arn arn:aws:sns:us-east-1:000000000000:cow-alerts --message "Cow 101 has fever!"
```

.NET Developer:

```
var sns = new AmazonSimpleNotificationServiceClient(creds, new AmazonSimpleNotificationServiceConfig { ServiceURL="http://localhost:4566" });
var topic = await sns.CreateTopicAsync("cow-alerts");
await sns.PublishAsync(topic.TopicArn, "Cow 101 has fever!");
```

Project Example: Cow ka temperature zyada hua → SNS ne SMS bheja farmer ko + email bheja vet ko.

◆ 4. SES — Simple Email Service

Kya hai? Email bhejne ka service — bulk emails, transactional emails.

Real-Life Example: Post office ka **speed post** — fast, reliable, tracked.

AWS Equivalent: AWS SES

Floci Local Usage:

```
aws --endpoint-url=http://localhost:4566 ses verify-email-identity --email-address
farm@example.com
aws --endpoint-url=http://localhost:4566 ses send-email --from farm@example.com --
destination "ToAddresses=test@example.com" --message
"Subject={Data=Alert},Body={Text={Data=Cow needs attention}}"
```

.NET Developer:

```
var ses = new AmazonSimpleEmailServiceClient(creds, new
    AmazonSimpleEmailServiceConfig { ServiceURL="http://localhost:4566" });
var req = new SendEmailRequest("farm@example.com", new Destination(new
    List<string>{"farmer@test.com"}), new Message(new Content("Alert"), new
    Body(new Content("Cow temperature high"))));
await ses.SendEmailAsync(req);
```

Project Example: Daily report farmer ke email pe automatic bhejo.

◆ 5. DynamoDB — NoSQL Database

Kya hai? Fast, schema-less database. Koi fixed table structure nahi — har item alag ho sakta hai.

Real-Life Example: WhatsApp notes — jaisa man kare likho, koi format nahi. SQL = School register (fixed columns).

AWS Equivalent: AWS DynamoDB

Floci Local Usage:

```
aws --endpoint-url=http://localhost:4566 dynamodb create-table --table-name cow-
readings --attribute-definitions AttributeName=CowId,AttributeType=S --key-
schema AttributeName=CowId,KeyType=HASH --billing-mode PAY_PER_REQUEST
aws --endpoint-url=http://localhost:4566 dynamodb put-item --table-name cow-readings
--item '{"CowId":{"S":"cow-001"},"Temp":{"N":"39.5"}}'
```

.NET Developer:

```
var ddb = new AmazonDynamoDBClient(creds, new AmazonDynamoDBConfig {
    ServiceURL="http://localhost:4566" });
await ddb.PutItemAsync("cow-readings", new Dictionary<string, AttributeValue>{
    ["CowId"] = new AttributeValue{S="cow-001"},
    ["Temp"] = new AttributeValue{N="39.5"}
});
```

Project Example: IoT sensor se cow ka temperature har minute DynamoDB mein store karo.

◆ 6. DynamoDB Streams

Kya hai? DynamoDB table mein kuch bhi change hua (insert/update/delete) → automatic event stream.

Real-Life Example: Bank account mein paisa aaya/gaya → SMS alert automatic. Stream = real-time ledger watcher.

AWS Equivalent: AWS DynamoDB Streams

Floci Local Usage:

```
aws --endpoint-url=http://localhost:4566 dynamodb create-table --table-name cow-
  readings --attribute-definitions AttributeName=CowId,AttributeType=S --key-
  schema AttributeName=CowId,KeyType=HASH --billing-mode PAY_PER_REQUEST --
  stream-specification StreamEnabled=true,StreamViewType=NEW_IMAGE
# Stream ARN use karke Lambda trigger karo
```

.NET Developer: .NET mein Lambda trigger se handle karte hain. Stream se events nikaal ke Lambda process karta hai.

Project Example: Naya sensor reading aaya → Stream ne pakda → Lambda ne alert check kiya.

◆ 7. Lambda — Serverless Functions

Kya hai? Code bina server ke chalao. Event aaya → code chala. 2 second mein deploy, instant run.

Real-Life Example: Ghar ka **automatic sensor light** — aadmi aaya, light apne aap on. Lambda = aadmi, trigger = sensor.

AWS Equivalent: AWS Lambda

Floci Local Usage:

```
# .NET Lambda deploy karo
aws --endpoint-url=http://localhost:4566 lambda create-function --function-name
  ProcessCow --runtime dotnet8 --handler
  CowApp::CowApp.Function::FunctionHandler --zip-file fileb://deployment.zip --
  role arn:aws:iam::000000000000:role/lambda-role
```

.NET Developer:

```
// Lambda function code (.NET)
public class Function
{
    public string FunctionHandler(SQSEvent evnt, ILambdaContext context)
    {
        foreach(var msg in evnt.Records)
        {
            Console.WriteLine($"Processing: {msg.Body}");
        }
        return "Done";
    }
}
```

Project Example: S3 mein cow image aayi → Lambda ne AI analysis ki → result DynamoDB mein dala.

◆ 8. API Gateway v1 — REST APIs

Kya hai? Traditional REST API ka **gatekeeper**. HTTP methods (GET, POST, PUT, DELETE) handle karta hai.

Real-Life Example: Building ka **receptionist + security guard** — har visitor ko check karo, sahi office mein bhejo.

AWS Equivalent: AWS API Gateway REST API

Floci Local Usage:

```
aws --endpoint-url=http://localhost:4566 apigateway create-rest-api --name CowApi
aws --endpoint-url=http://localhost:4566 apigateway create-resource --rest-api-id
    abc123 --parent-id xyz --path-part cows
```

.NET Developer:

```
// .NET mein API Gateway usually direct interact nahi karte
// Instead, .NET API Gateway pe deploy hota hai
// Development mein bas Swagger se test karo: http://localhost:5000/swagger
// Floci API Gateway testing ke liye use hota hai
```

Project Example: Mobile app se GET /cows/001 → API Gateway → Lambda → DynamoDB se data.

◆ 9. API Gateway v2 — HTTP/WebSocket APIs

Kya hai? Modern, fast, cheap HTTP APIs + **WebSocket** (real-time chat/live updates).

Real-Life Example: WhatsApp group = WebSocket (real-time messages).
Normal SMS = HTTP API v1.

AWS Equivalent: AWS API Gateway HTTP API / WebSocket API

Floci Local Usage:

```
aws --endpoint-url=http://localhost:4566 apigatewayv2 create-api --name CowHttpApi --
    protocol-type HTTP
aws --endpoint-url=http://localhost:4566 apigatewayv2 create-api --name CowWebSocket
    --protocol-type WEBSOCKET
```

.NET Developer:

```
// .NET ke SignalR ya WebSocket handlers ko API Gateway v2 WebSocket se connect kar
    sakte hain
// Real-time cow location tracking ke liye best hai
```

Project Example: Farmer ke mobile pe cow ka location real-time dikhe (WebSocket) via API Gateway v2.

◆ 10. Cognito — User Login & Authentication

Kya hai? User sign-up, sign-in, password reset, JWT token generation. “Kaun hai tu?” prove karta hai.

Real-Life Example: Cinema hall ka **ticket checker** — valid ticket (JWT token) hai toh andar, nahi toh nikal!

AWS Equivalent: AWS Cognito (User Pools + Identity Pools)

Floci Local Usage:

```
aws --endpoint-url=http://localhost:4566 cognito-idp create-user-pool --pool-name FarmerPool
aws --endpoint-url=http://localhost:4566 cognito-idp sign-up --client-id abc123 --username farmer1 --password Password123!
aws --endpoint-url=http://localhost:4566 cognito-idp initiate-auth --client-id abc123 --auth-flow USER_PASSWORD_AUTH --auth-parameters USERNAME=farmer1,PASSWORD=Password123!
```

.NET Developer:

```
// .NET API mein JWT validation
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options => {
        options.Authority = "http://localhost:4566/us-east-1_abc123"; // Floci Cognito
        options.TokenValidationParameters = new TokenValidationParameters {
            ValidateAudience = false };
    });
```

Project Example: Farmer login → JWT token mila → .NET API se cow data access.

◆ 11. IAM — Identity & Access Management

Kya hai? “Kaun kya kar sakta hai?” Users, roles, policies manage karta hai.

Real-Life Example: Office ka **access card system** — CEO (har jagah), Employee (apna floor), Peon (sirf store).

AWS Equivalent: AWS IAM

Floci Local Usage:

```
aws --endpoint-url=http://localhost:4566 iam create-role --role-name CowLambdaRole --assume-role-policy-document file://trust-policy.json
aws --endpoint-url=http://localhost:4566 iam attach-role-policy --role-name CowLambdaRole --policy-arn arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess
```

.NET Developer: .NET mein IAM usually direct interact nahi karte — ye backend setup hai. SDK credentials through IAM role milti hain.

Project Example: Lambda role sirf S3 read aur DynamoDB write kar sake — baki sab block.

◆ 12. STS — Security Token Service

Kya hai? Temporary credentials generate karta hai — short-term access. Risk kam hota hai.

Real-Life Example: Hotel ka **temporary visitor pass** — 2 din valid, phir expire. Permanent employee card nahi dena!

AWS Equivalent: AWS STS

Floci Local Usage:

```
aws --endpoint-url=http://localhost:4566 sts assume-role --role-arn
    arn:aws:iam::000000000000:role/CowTempRole --role-session-name FarmerSession
```

.NET Developer:

```
var sts = new AmazonSecurityTokenServiceClient(creds, new
    AmazonSecurityTokenServiceConfig { ServiceURL="http://localhost:4566" });
var token = await sts.AssumeRoleAsync(new AssumeRoleRequest {
    RoleArn = "arn:aws:iam::000000000000:role/CowTempRole",
    RoleSessionName = "FarmerSession",
    DurationSeconds = 3600
});
// token.Credentials se temporary access key mila
```

Project Example: Third-party app ko 1 hour ke liye S3 access do — baad mein automatically band.

◆ 13. KMS — Key Management Service

Kya hai? Encryption keys ka manager. Sensitive data ko **lock** karne ke liye.

Real-Life Example: Ghar ka **locker key + main door key** — KMS = key factory jo secure keys banata hai.

AWS Equivalent: AWS KMS

Floci Local Usage:

```
aws --endpoint-url=http://localhost:4566 kms create-key --description
    "CowDataEncryption"
aws --endpoint-url=http://localhost:4566 kms encrypt --key-id alias/cow-key --
    plaintext "SensitiveCowData"
```


.NET Developer:

```
var kms = new AmazonKeyManagementServiceClient(creds, new
    AmazonKeyManagementServiceConfig { ServiceURL="http://localhost:4566" });
var enc = await kms.EncryptAsync(new EncryptRequest { KeyId="alias/cow-key",
    Plaintext=new MemoryStream(Encoding.UTF8.GetBytes("Aadhar1234")) });
var cipher = Convert.ToBase64String(enc.CiphertextBlob.ToArray());
```

Project Example: Cow owner ka Aadhar number encrypt kar ke DynamoDB mein store karo.

◆ 14. Secrets Manager

Kya hai? Passwords, API keys, tokens ko **secure vault** mein store karta hai. Code mein hardcode nahi!

Real-Life Example: Ghar ki **tijori (safe)** — cheque, jewellery. Almari mein nahi, tijori mein!

AWS Equivalent: AWS Secrets Manager

Floci Local Usage:

```
aws --endpoint-url=http://localhost:4566 secretsmanager create-secret --name
    CowDBPassword --secret-string "SuperSecret123!"
aws --endpoint-url=http://localhost:4566 secretsmanager get-secret-value --secret-id
    CowDBPassword
```

.NET Developer:

```
var sm = new AmazonSecretsManagerClient(creds, new AmazonSecretsManagerConfig {
    ServiceURL="http://localhost:4566" });
var secret = await sm.GetSecretValueAsync(new GetSecretValueRequest { SecretId =
    "CowDBPassword" });
var password = secret.SecretString; // "SuperSecret123!"
```

Project Example: Database password, payment gateway API key — sab Secrets Manager mein rakho.

◆ 15. SSM Parameter Store

Kya hai? Application configuration store — environment variables, settings, feature flags.

Real-Life Example: Restaurant ka **menu board** — prices, items, offers — ek jagah likhe, sab dekh sakein.

AWS Equivalent: AWS Systems Manager Parameter Store

Floci Local Usage:

```
aws --endpoint-url=http://localhost:4566 ssm put-parameter --name "/cow-app/max-temp" --value "39.5" --type String
aws --endpoint-url=http://localhost:4566 ssm get-parameter --name "/cow-app/max-temp"
```

.NET Developer:

```
var ssm = new AmazonSimpleSystemsManagementClient(creds, new
    AmazonSimpleSystemsManagementConfig { ServiceURL="http://localhost:4566" });
var param = await ssm.GetParameterAsync(new GetParameterRequest { Name = "/cow-app/max-temp" });
var maxTemp = decimal.Parse(param.Parameter.Value); // 39.5
```

Project Example: Alert threshold 39.5°C Parameter Store mein rakho — badalna hai toh ek jagah change karo.

◆ 16. ACM — AWS Certificate Manager

Kya hai? SSL/TLS certificates banata aur manage karta hai — HTTPS secure connections ke liye.

Real-Life Example: Bank ka **seal** — asli hai toh trust karo, nakli hai toh door raho.

AWS Equivalent: AWS ACM

Floci Local Usage:

```
aws --endpoint-url=http://localhost:4566 acm request-certificate --domain-name
    farm.example.com --validation-method DNS
```

.NET Developer: .NET apps mein ACM certificates usually load balancer/API Gateway pe lagte hain. Local development mein `https://localhost:5001` (dev cert) kaafi hai.

Project Example: Production mein `https://farm.example.com` pe valid SSL certificate.

◆ 17. EventBridge — Event Bus & Rules

Kya hai? Central event bus. Events aaye → rules match hui → target pe bhej do.

Real-Life Example: Office ka **intercom system** — reception pe call aayi → rule: "IT ka call" → IT department.

AWS Equivalent: AWS EventBridge

Floci Local Usage:

```
aws --endpoint-url=http://localhost:4566 events create-event-bus --name cow-events
aws --endpoint-url=http://localhost:4566 events put-rule --name high-temp-rule --
    event-pattern '{"source":["cow.sensor"],"detail-type":
    ["temperature.reading"]}'
aws --endpoint-url=http://localhost:4566 events put-targets --rule high-temp-rule --
    targets Id=1,Arn=arn:aws:lambda:us-east-1:000000000000:function:AlertLambda
```

.NET Developer:

```
var eb = new AmazonEventBridgeClient(creds, new AmazonEventBridgeConfig {
    ServiceURL="http://localhost:4566" });
await eb.PutEventsAsync(new PutEventsRequest {
    Entries = new List<PutEventsRequestEntry> {
        new PutEventsRequestEntry { Source="cow.sensor",
            DetailType="temperature.reading", Detail="{\"cowId\": \"001\",
            \"temp\": 40.2}" }
    }
});
```

Project Example: Sensor event aaya → EventBridge ne rule check ki → SNS pe alert bheja.

◆ 18. EventBridge Pipes

Kya hai? Direct pipeline — event source (SQS, DynamoDB Stream) se target (Lambda, SNS) mein bina rule likhe.

Real-Life Example: Pipeline — ek tank se dusre tank mein paise bina bucket ke — direct flow.

AWS Equivalent: AWS EventBridge Pipes

Floci Local Usage:

```
aws --endpoint-url=http://localhost:4566 pipes create-pipe --name cow-pipe --source
    arn:aws:sqs:us-east-1:000000000000:cow-queue --target arn:aws:lambda:us-
    east-1:000000000000:function:ProcessCow
```

.NET Developer: Pipes configuration ke through setup hota hai — .NET code se direct interact kam hota hai.

Project Example: SQS queue se direct Lambda trigger — bina EventBridge rules likhe.

◆ 19. EventBridge Scheduler

Kya hai? Time-based jobs — har ghante, roz, mahine mein ek baar — scheduled tasks.

Real-Life Example: Alarm clock — roz subah 6 baje bajta hai. Scheduler = programmable alarm.

AWS Equivalent: AWS EventBridge Scheduler

Floci Local Usage:

```
aws --endpoint-url=http://localhost:4566 scheduler create-schedule --name daily-report --schedule-expression "rate(1 day)" --target '{"RoleArn":"arn:aws:iam::000000000000:role/scheduler","Arn":"arn:aws:lambda:us-east-1:000000000000:function:DailyReport"}'
```

.NET Developer: Scheduler ke targets mein Lambda functions ya API endpoints hote hain — .NET Lambda function banake schedule karo.

Project Example: Roz raat 11 baje daily cow health report generate ho.

◆ 20. Step Functions

Kya hai? Step-by-step workflow automation. Decision tree — agar A hua toh B karo, nahi toh C.

Real-Life Example: Hospital patient flow: Reception → Checkup → Test → Report → Medicine.

AWS Equivalent: AWS Step Functions

Floci Local Usage:

```
aws --endpoint-url=http://localhost:4566 stepfunctions create-state-machine --name cow-workflow --definition file://workflow.json --role-arn arn:aws:iam::000000000000:role/stepfunctions
```

.NET Developer:

```
// Step Functions SDK se workflow start karo
var sfn = new AmazonStepFunctionsClient(creds, new AmazonStepFunctionsConfig {
    ServiceURL="http://localhost:4566" });
await sfn.StartExecutionAsync(new StartExecutionRequest {
    StateMachineArn = "arn:aws:states:us-east-1:000000000000:stateMachine:cow-workflow",
    Input = "{\"cowId\":\"001\",\"temp\":40.5}"
});
```

Project Example: 1. Temperature check → 2. Agar > 39.5 → 3. SMS bhejo → 4. Vet ko assign → 5. Follow-up queue.

◆ 21. Kinesis — Data Streaming

Kya hai? Real-time data streams. IoT, logs, clickstreams — massive real-time data handle karta hai.

Real-Life Example: Highway pe toll plaza — har gaadi ka data real-time stream mein ja raha hai.

AWS Equivalent: Amazon Kinesis Data Streams

Floci Local Usage:

```
aws --endpoint-url=http://localhost:4566 kinesis create-stream --stream-name cow-sensor-stream --shard-count 1
aws --endpoint-url=http://localhost:4566 kinesis put-record --stream-name cow-sensor-stream --partition-key cow-001 --data "eyJ0ZW1wIjogMzkuNX0="
```

.NET Developer:

```
var kinesis = new AmazonKinesisClient(creds, new AmazonKinesisConfig {
    ServiceURL="http://localhost:4566" });
await kinesis.PutRecordAsync(new PutRecordRequest {
    StreamName = "cow-sensor-stream",
    PartitionKey = "cow-001",
    Data = new MemoryStream(Encoding.UTF8.GetBytes("{\"temp\":39.5"}))
});
```

Project Example: 10,000 cows se real-time sensor data stream karo — analytics ke liye.

◆ 22. Data Firehose

Kya hai? Stream ko automatically destination pe delivery karta hai — S3, Elasticsearch, Redshift.

Real-Life Example: Pipeline jo stream se paise direct storage tank mein bhar de.

AWS Equivalent: Amazon Kinesis Data Firehose

Floci Local Usage:

```
aws --endpoint-url=http://localhost:4566 firehose create-delivery-stream --delivery-stream-name cow-firehose --delivery-stream-type DirectPut --s3-destination-configuration RoleARN=arn:aws:iam::000000000000:role/firehose,BucketARN=arn:aws:s3:::cow-logs
```

.NET Developer: Firehose usually Kinesis se connected hota hai — .NET mein direct Firehose pe bhi data bhej sakte hain.

Project Example: Sensor logs Firehose se S3 mein automatic archive hote rahein.

◆ 23. RDS — Relational Database Service

Kya hai? SQL databases — PostgreSQL, MySQL, MariaDB. Tables, rows, relationships — traditional DB.

Real-Life Example: Excel sheet — sab kuch organized, fixed columns, har row ek customer/order.

AWS Equivalent: AWS RDS

Floci Local Usage:

```
# PostgreSQL container Docker mein alag se chalo (Floci ke saath)
# docker-compose.yml mein already setup hai!
docker exec -it my-local-postgres psql -U admin -d myfarmdb -c
    "CREATE TABLE cows (id SERIAL PRIMARY KEY, name VARCHAR(50), breed
    VARCHAR(50));"
```

.NET Developer:

```
// appsettings.json
"ConnectionStrings": { "Default":
    "Host=localhost;Port=5432;Database=myfarmdb;Username=admin;Password=admin123" }

// Entity Framework Core
public class Cow { public int Id { get; set; } public string Name { get; set; }
    public string Breed { get; set; } }
public class FarmDbContext : DbContext { public DbSet<Cow> Cows { get; set; } }
```

Project Example: Customer orders, payment records, cow breeding history — sab RDS mein.

◆ 24. ElastiCache — Redis-style Cache

Kya hai? Super-fast in-memory cache. Database se baar-baar same data na nikale — cache mein rakho.

Real-Life Example: Table drawer — roz use hone wali cheezein drawer mein (fast), baaki almari mein (slow).

AWS Equivalent: Amazon ElastiCache (Redis/Memcached)

Floci Local Usage:

```
aws --endpoint-url=http://localhost:4566 elasticache create-cache-cluster --cache-
cluster-id cow-cache --engine redis --num-cache-nodes 1
```

.NET Developer:

```
// StackExchange.Redis NuGet package
var redis = ConnectionMultiplexer.Connect("localhost:6379");
var db = redis.GetDatabase();
await db.StringSetAsync("cow:001:temp", "39.5", TimeSpan.FromMinutes(5));
var temp = await db.StringGetAsync("cow:001:temp");
```

Project Example: Cow ka latest temperature cache mein rakho — 5 minute tak DB se na puchho.

◆ 25. MSK / Kafka — Kafka Messaging

Kya hai? Apache Kafka — distributed messaging system. High-throughput, fault-tolerant event streaming.

Real-Life Example: News agency — har reporter se news aayi, central office distribute karta hai sab channels ko.

AWS Equivalent: Amazon MSK (Managed Streaming for Kafka)

Floci Local Usage:

```
aws --endpoint-url=http://localhost:4566 kafka create-cluster --cluster-name cow-  
kafka --kafka-version 2.8.1 --broker-node-group-info  
InstanceType=kafka.m5.large
```

.NET Developer:

```
// Confluent.Kafka NuGet  
var config = new ProducerConfig { BootstrapServers = "localhost:9092" };  
using var producer = new ProducerBuilder<Null, string>(config).Build();  
await producer.ProduceAsync("cow-topic", new Message<Null, string> { Value =  
    "{\"temp\":39.5}" });
```

Project Example: Multiple farms se data Kafka mein — ek central analytics system consume kare.

◆ 26. OpenSearch — Search & Log Analytics

Kya hai? Search engine + analytics. Logs search karo, dashboards banao, full-text search.

Real-Life Example: Library ka index card system — koi bhi book 2 second mein dhoondo.

AWS Equivalent: Amazon OpenSearch Service (Elasticsearch ka AWS version)

Floci Local Usage:

```
aws --endpoint-url=http://localhost:4566 opensearch create-domain --domain-name cow-  
logs
```

.NET Developer:

```
// NEST (Elasticsearch .NET client)  
var client = new ElasticClient(new ConnectionSettings(new Uri("http://  
localhost:4566")));  
await client.IndexDocumentAsync(new CowLog { CowId = "001", Message = "Temperature  
alert", Timestamp = DateTime.Now });
```

Project Example: 1 lakh cow logs search karo — “kitni cows ko fever tha last week?”

◆ 27. ECR — Elastic Container Registry

Kya hai? Docker images ka store. Aapka built .NET app image yahan store hota hai.

Real-Life Example: Photo album — par sirf Docker images ke liye! 😊

AWS Equivalent: Amazon ECR

Floci Local Usage:

```
aws --endpoint-url=http://localhost:4566 ecr create-repository --repository-name cow-api
aws --endpoint-url=http://localhost:4566 ecr put-image --repository-name cow-api --image-tag latest --image-manifest file://manifest.json
```

.NET Developer:

```
# Dockerfile se image banao
dotnet publish -c Release
docker build -t cow-api:latest .

# Floci ECR mein push karo (local testing ke liye)
aws --endpoint-url=http://localhost:4566 ecr put-image ...
```

Project Example: .NET API ka Docker image ECR mein store karo — ECS pe deploy karo.

◆ 28. ECS — Elastic Container Service

Kya hai? Docker containers ko run karo, scale karo, manage karo — AWS ka container orchestrator.

Real-Life Example: Bus depot — buses (containers) ko schedule karo, route do, maintain karo.

AWS Equivalent: Amazon ECS (Fargate / EC2 launch type)

Floci Local Usage:

```
aws --endpoint-url=http://localhost:4566 ecs create-cluster --cluster-name cow-cluster
aws --endpoint-url=http://localhost:4566 ecs register-task-definition --family cow-api-task --container-definitions '[{"name":"cow-api","image":"cow-api:latest","memory":512,"portMappings":[{"containerPort":8080}]}]'
```

.NET Developer: .NET app ko Dockerize karo → ECR mein daalo → ECS task definition banao → run karo.

Project Example: .NET API ko ECS pe 3 replicas chalao — load balancing ke saath.

◆ 29. EKS — Elastic Kubernetes Service

Kya hai? Managed Kubernetes. Containers ko **pod** mein, **deployment** mein, **service** mein manage karo.

Real-Life Example: Army ka hierarchy — Soldier (container) → Squad (pod) → Battalion (deployment) → Division (cluster).

AWS Equivalent: Amazon EKS

Floci Local Usage:

```
aws --endpoint-url=http://localhost:4566 eks create-cluster --name cow-k8s --role-arn arn:aws:iam::000000000000:role/eks-role --resources-vpc-config subnetIds=subnet-123,securityGroupIds=sg-123
```

.NET Developer:

```
# deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: cow-api
spec:
  replicas: 3
  selector:
    matchLabels:
      app: cow-api
  template:
    metadata:
      labels:
        app: cow-api
    spec:
      containers:
        - name: cow-api
          image: cow-api:latest
          ports:
            - containerPort: 8080
```

Project Example: .NET API ko Kubernetes pe deploy karo — auto-scaling, self-healing!

◆ 30. EC2 — Elastic Compute Cloud

Kya hai? Virtual machines (VMs). Windows/Linux server cloud pe.

Real-Life Example: Rent pe computer — apna nahi, par pura control hai. Band karo jab man kare.

AWS Equivalent: AWS EC2

Floci Local Usage:

```
aws --endpoint-url=http://localhost:4566 ec2 run-instances --image-id ami-12345 --  
instance-type t2.micro --key-name my-key
```

.NET Developer: .NET apps usually Lambda/ECS/EKS pe chalte hain — EC2 pe legacy apps ya custom server requirements ke liye.

Project Example: Windows Server pe .NET Framework 4.8 app chalani hai — EC2 pe VM le lo.

◆ 31. Auto Scaling

Kya hai? Load badhe toh servers automatic badhe — load kam hua toh servers automatic kam.

Real-Life Example: Restaurant mein waiters — rush mein 10 waiters, shaanti mein 3 waiters.

AWS Equivalent: AWS Auto Scaling / EC2 Auto Scaling Groups

Floci Local Usage:

```
aws --endpoint-url=http://localhost:4566 autoscaling create-auto-scaling-group --  
auto-scaling-group-name cow-asg --min-size 1 --max-size 5 --desired-capacity  
2
```

.NET Developer: ECS/EKS ke saath auto scaling configure karte hain — .NET containers automatically scale hote hain.

Project Example: Cow monitoring app — subah 1000 users → 5 containers, raat ko 10 users → 1 container.

◆ 32. ELB v2 — Elastic Load Balancing

Kya hai? Traffic ko multiple servers mein baanto — koi ek server overload na ho.

Real-Life Example: Bank ke multiple counters — line mein 20 log hain → 4 counters mein baant do.

AWS Equivalent: AWS ALB (Application Load Balancer) / NLB (Network Load Balancer)

Floci Local Usage:

```
aws --endpoint-url=http://localhost:4566 elbv2 create-load-balancer --name cow-alb --  
subnets subnet-1 --security-groups sg-1  
aws --endpoint-url=http://localhost:4566 elbv2 create-target-group --name cow-tg --  
protocol HTTP --port 8080 --vpc-id vpc-123
```

.NET Developer: .NET API ko ECS pe chalao → ALB traffic distribute karega → healthy containers pe request jayegi.

Project Example: 3 .NET API containers — ALB decide karega kaunsa container request handle kare.

◆ 33. CloudFormation

Kya hai? Infrastructure as Code — YAML/JSON mein likho: “Mujhe ye resources chahiye” → ek command se sab ban jaye.

Real-Life Example: Lego instruction manual — step-by-step, ek baar follow karo → perfect building.

AWS Equivalent: AWS CloudFormation

Floci Local Usage:

```
aws --endpoint-url=http://localhost:4566 cloudformation create-stack --stack-name  
cow-infra --template-body file://infra.yaml
```

.NET Developer:

```
# infra.yaml snippet  
Resources:  
  CowBucket:  
    Type: AWS::S3::Bucket  
    Properties:  
      BucketName: cow-images-bucket  
  CowTable:  
    Type: AWS::DynamoDB::Table  
    Properties:  
      TableName: cow-readings  
      BillingMode: PAY_PER_REQUEST  
      AttributeDefinitions:  
        - AttributeName: CowId  
          AttributeType: S  
      KeySchema:  
        - AttributeName: CowId  
          KeyType: HASH
```

Project Example: Ek YAML file se pura infrastructure setup — S3, DynamoDB, Lambda, API Gateway.

◆ 34. CodeBuild

Kya hai? Code ko compile, test, build karo — automated CI pipeline.

Real-Life Example: Factory assembly line — parts aaye, assemble hua, test hua, final product ready.

AWS Equivalent: AWS CodeBuild

Floci Local Usage:

```
aws --endpoint-url=http://localhost:4566 codebuild create-project --name cow-build --
source type=S3,location=cow-bucket/source.zip --artifacts
type=S3,location=cow-bucket/build.zip --environment
type=LINUX_CONTAINER,image=aws/codebuild/
standard:5.0,computeType=BUILD_GENERAL1_SMALL
```

.NET Developer:

```
# buildspec.yml
version: 0.2
phases:
  build:
    commands:
      - dotnet restore
      - dotnet build
      - dotnet test
      - dotnet publish -c Release -o ./publish
artifacts:
  files:
    - ./publish/**/*
```

Project Example: Push karo GitHub pe → CodeBuild automatically test aur build kare → artifact S3 mein.

◆ 35. CodeDeploy

Kya hai? Build hua code servers pe deploy karo — bina downtime ke (blue/green deployment).

Real-Life Example: Plane mein fuel change — udti plane mein purana fuel khatam, naya fuel aa gaya — landing nahi!

AWS Equivalent: AWS CodeDeploy

Floci Local Usage:

```
aws --endpoint-url=http://localhost:4566 deploy create-application --application-
name cow-api-app
aws --endpoint-url=http://localhost:4566 deploy create-deployment-group --
application-name cow-api-app --deployment-group-name prod --service-role-arn
arn:aws:iam::000000000000:role/codedeploy
```

.NET Developer: CodeDeploy ECS/EC2 pe .NET container deploy karta hai — zero downtime.

Project Example: New version aaya → old containers chalte rahe → new containers ready → switch!

◆ 36. AppConfig

Kya hai? Application configuration manage karo — feature flags, configuration profiles. Deploy bina code change ke.

Real-Life Example: AC remote — temperature badal sakte ho bina AC khole.

AWS Equivalent: AWS AppConfig

Floci Local Usage:

```
aws --endpoint-url=http://localhost:4566 appconfig create-application --name cow-app
aws --endpoint-url=http://localhost:4566 appconfig create-configuration-profile --
    application-id abc123 --name cow-config --location-uri "ssm:/cow-app/config"
```

.NET Developer:

```
// AppConfig se dynamic configuration fetch karo
// Feature flag: "new-dashboard-enabled" → true/false
```

Project Example: Naya dashboard feature 10% users pe test karo — flag se control karo.

◆ 37. AppConfigData

Kya hai? Runtime mein application configuration access karo — AppConfig se connected.

Real-Life Example: Phone ki settings app — real-time mein settings change karo.

AWS Equivalent: AWS AppConfig Data API

Floci Local Usage:

```
aws --endpoint-url=http://localhost:4566 appconfigdata start-configuration-session --
    application-identifier cow-app --environment-identifier prod --configuration-
    profile-identifier cow-config
```

.NET Developer: .NET app startup pe configuration fetch karo — runtime mein refresh karo.

◆ 38. Resource Groups Tagging API

Kya hai? AWS resources ko tags lagao — manage karna, billing track karna, search karna easy ho jaata hai.

Real-Life Example: Library mein books ko color-coded stickers — red = fiction, blue = science.

AWS Equivalent: AWS Resource Groups Tagging API

Floci Local Usage:

```
aws --endpoint-url=http://localhost:4566 resourcegroupstaggingapi tag-resources --  
resource-arn-list arn:aws:s3:::cow-bucket --tags  
Environment=Local,Project=CowApp
```

.NET Developer: Usually DevOps/Admin ka kaam — .NET code se kam interact hota hai.

Project Example: S3 bucket pe tag `Environment=Production` → cost report mein alag dikhe.

◆ 39. CloudWatch Logs

Kya hai? Application logs collect, store, search karo. `Console.WriteLine` ke liye central place.

Real-Life Example: CCTV recording — kya hua, kab hua, sab recorded.

AWS Equivalent: Amazon CloudWatch Logs

Floci Local Usage:

```
aws --endpoint-url=http://localhost:4566 logs create-log-group --log-group-name /cow/  
api  
aws --endpoint-url=http://localhost:4566 logs create-log-stream --log-group-name /  
cow/api --log-stream-name api-server-1  
aws --endpoint-url=http://localhost:4566 logs put-log-events --log-group-name /cow/  
api --log-stream-name api-server-1 --log-events  
timestamp=1716400000000,message="Cow API started"
```

.NET Developer:

```
// Serilog + AWS CloudWatch sink  
Log.Logger = new LoggerConfiguration()  
    .WriteTo.AmazonCloudWatchLogs("/cow/api", "api-1", new  
        BasicAWSCredentials("test","test"), new AmazonCloudWatchLogsConfig {  
            ServiceURL="http://localhost:4566" })  
    .CreateLogger();  
Log.Information("Cow {CowId} temperature recorded: {Temp}", cowId, temp);
```

Project Example: .NET API ke saare logs CloudWatch mein — “last week kitni errors aayi?”

◆ 40. CloudWatch Metrics

Kya hai? Numbers track karo — CPU usage, request count, error rate, cow temperature average.

Real-Life Example: Car dashboard — speed, fuel, temperature — numbers se health pata chalti hai.

AWS Equivalent: Amazon CloudWatch Metrics

Floci Local Usage:

```
aws --endpoint-url=http://localhost:4566 cloudwatch put-metric-data --namespace CowApp --metric-data MetricName=CowTemp,Value=39.5,Unit=None
```

.NET Developer:

```
var cw = new AmazonCloudWatchClient(creds, new AmazonCloudWatchConfig {
    ServiceURL="http://localhost:4566" });
await cw.PutMetricDataAsync(new PutMetricDataRequest {
    Namespace = "CowApp",
    MetricData = new List<MetricDatum> {
        new MetricDatum { MetricName = "CowTemp", Value = 39.5m, Unit =
            StandardUnit.None, Dimensions = new List<Dimension> { new Dimension {
                Name="CowId", Value="001" } } }
    }
});
```

Project Example: Dashboard pe graph dikhe — “Sab cows ka average temperature din bhar kya raha?”

◆ 41. AWS Config

Kya hai? AWS resources ka configuration track karo. Kya change hua, kab hua — pura history.

Real-Life Example: Property registry office — kisne kab kya property kharidi/beche — sab record.

AWS Equivalent: AWS Config

Floci Local Usage:

```
aws --endpoint-url=http://localhost:4566 configservice put-config-rule --config-rule name=required-tags,source={Owner=AWS,SourceIdentifier=REQUIRED_TAGS}
```

.NET Developer: Usually compliance/security team use karti hai — .NET code se direct interact kam.

Project Example: S3 bucket bina encryption ke create hua → Config ne alert kiya.

◆ 42. AWS Backup

Kya hai? Automated backup policies — daily, weekly, monthly backup schedule.

Real-Life Example: Photo ka Google Drive backup — roz raat ko automatic sync.

AWS Equivalent: AWS Backup

Floci Local Usage:

```
aws --endpoint-url=http://localhost:4566 backup create-backup-plan --backup-plan
BackupPlanName=cow-
backup,Rules=[{RuleName=daily,TargetBackupVaultName=cow-vault,ScheduleExpression="cron(0
5 ? * * *)"}]]
```

.NET Developer: Backup policies setup through AWS Console/CLI — .NET app se direct interaction kam.

Project Example: Cow data roz subah 5 baje automatic backup — 30 days retain karo.

◆ 43. Glue — Data Catalog

Kya hai? Data lake ka catalog — kahan ka data hai, kya format hai, kaun use kar sakta hai.

Real-Life Example: Library ka master catalog — kis shelf pe kya hai, kis section mein hai.

AWS Equivalent: AWS Glue Data Catalog

Floci Local Usage:

```
aws --endpoint-url=http://localhost:4566 glue create-database --database-input
Name=cow-data,Description="Cow sensor data catalog"
aws --endpoint-url=http://localhost:4566 glue create-table --database-name cow-data
--table-input Name=cow-
readings,StorageDescriptor={Columns=[{Name=cow_id,Type=string},
{Name=temp,Type=double}]}
```

.NET Developer: .NET mein usually Athena queries ke saath Glue catalog use hota hai.

Project Example: 1 year ka cow data — Glue catalog mein schema define → Athena se query karo.

◆ 44. Athena

Kya hai? S3 mein stored data pe SQL query chalao — bina database load kiye!

Real-Life Example: Warehouse mein kisi bhi box pe magnifying glass se dekhna — bina box khole!

AWS Equivalent: Amazon Athena

Floci Local Usage:

```
aws --endpoint-url=http://localhost:4566 athena start-query-execution --query-string  
"SELECT cow_id, AVG(temp) FROM cow_readings GROUP BY cow_id" --work-group  
primary
```

.NET Developer:

```
var athena = new AmazonAthenaClient(creds, new AmazonAthenaConfig {  
    ServiceURL="http://localhost:4566" });  
var result = await athena.StartQueryExecutionAsync(new StartQueryExecutionRequest {  
    QueryString = "SELECT * FROM cow_data.cow_readings WHERE temp > 39.5",  
    WorkGroup = "primary"  
});
```

Project Example: S3 mein CSV files → Athena se SQL query → “Kaunsi cows ko frequently fever hota hai?”

◆ 45. Bedrock Runtime — AI Model Stub

Kya hai? AWS ke AI/ML models (Claude, Stable Diffusion) ko call karo. Floci mein stub (dummy response) deta hai.

Real-Life Example: Genius friend — kuch bhi poochho, smart jawab dega. (Floci mein yeh dummy hai!)

AWS Equivalent: Amazon Bedrock

Floci Local Usage:

```
aws --endpoint-url=http://localhost:4566 bedrock-runtime invoke-model --model-id  
anthropic.claude-v2 --body '{"prompt":"What is cow  
fever?","max_tokens_to_sample":200}'
```

.NET Developer:

```
var bedrock = new AmazonBedrockRuntimeClient(creds, new AmazonBedrockRuntimeConfig {  
    ServiceURL="http://localhost:4566" });  
var response = await bedrock.InvokeModelAsync(new InvokeModelRequest { ModelId =  
    "anthropic.claude-v2", Body = new  
    MemoryStream(Encoding.UTF8.GetBytes("{\"prompt\":\"What is cow fever?  
\"}\")) });
```

Project Example: Cow symptoms describe karo → AI se possible disease suggest karwao.

◆ 46. Textract

Kya hai? Documents se text, tables, forms automatically nikaalo. OCR on steroids.

Real-Life Example: Xerox machine jo padh bhi sake — paper pe likha text computer mein aa gaya.

AWS Equivalent: Amazon Textract

Floci Local Usage:

```
aws --endpoint-url=http://localhost:4566 textract detect-document-text --document
S3Object={Bucket=cow-docs,Name=vaccination.jpg}
```

.NET Developer:

```
var textract = new AmazonTextractClient(creds, new AmazonTextractConfig {
    ServiceURL="http://localhost:4566" });
var result = await textract.DetectDocumentTextAsync(new DetectDocumentTextRequest {
    Document = new Document { S3Object = new S3Object { Bucket = "cow-docs", Name =
        "vaccination.jpg" } }
});
foreach(var block in result.Blocks) Console.WriteLine(block.Text);
```

Project Example: Cow vaccination record photo → Textract se text nikala → RDS mein store.

◆ 47. Transcribe

Kya hai? Audio/video se text nikaalo — speech-to-text.

Real-Life Example: WhatsApp voice note ko text mein convert — bina sunne padhe padh lo!

AWS Equivalent: Amazon Transcribe

Floci Local Usage:

```
aws --endpoint-url=http://localhost:4566 transcribe start-transcription-job --
transcription-job-name cow-audio --media MediaFileUri=s3://cow-audio/farmer-
call.mp3 --language-code en-US
```

.NET Developer:

```
var transcribe = new AmazonTranscribeServiceClient(creds, new
    AmazonTranscribeServiceConfig { ServiceURL="http://localhost:4566" });
await transcribe.StartTranscriptionJobAsync(new StartTranscriptionJobRequest {
    TranscriptionJobName = "cow-audio-1",
    LanguageCode = LanguageCode.EnUS,
    Media = new Media { MediaFileUri = "s3://cow-audio/farmer-call.mp3" }
});
```

Project Example: Farmer ne voice message bheja cow ke symptoms ke baare mein → Transcribe se text → database.

◆ 48. CloudFront

Kya hai? Content Delivery Network (CDN). Static files (images, JS, CSS) fast deliver karo — world-wide edge servers se.

Real-Life Example: Pizza chain — har area mein outlet hai, delivery 5 minute mein. Central kitchen se 1 ghanta lagta.

AWS Equivalent: Amazon CloudFront

Floci Local Usage:

```
aws --endpoint-url=http://localhost:4566 cloudfront create-distribution --origin-domain-name cow-bucket.s3.localhost.localstack.cloud
```

.NET Developer:

```
// .NET API se CloudFront URL generate karo
var cdnUrl = $"https://d1234.cloudfront.localhost.localstack.cloud/{imageKey}";
// User ko CDN URL do — fast load hoga
```

Project Example: Cow ki 1000 photos — CloudFront se world-wide fast load honghi.

◆ 49. Route53

Kya hai? DNS service — domain name (farm.example.com) ko IP address (1.2.3.4) mein convert karo.

Real-Life Example: Phone book — naam (Ram) se number (9876543210) dhoondo.

AWS Equivalent: Amazon Route53

Floci Local Usage:

```
aws --endpoint-url=http://localhost:4566 route53 create-hosted-zone --name cow.farm --caller-reference 123
aws --endpoint-url=http://localhost:4566 route53 change-resource-record-sets --hosted-zone-id ABC123 --change-batch '{"Changes": [{"Action": "CREATE", "ResourceRecordSet": {"Name": "api.cow.farm", "Type": "A", "TTL": 300, "ResourceRecords": [{"Value": "127.0.0.1"}]}]}'
```

.NET Developer: .NET app mein direct use kam — DevOps/Admin configure karte hain.

Project Example: `api.cow.farm` → aapke .NET API pe point karo.

◆ 50. Transfer Family

Kya hai? Secure file transfer — SFTP, FTPS, FTP. Legacy systems se cloud pe files laana.

Real-Life Example: Courier service — secure package delivery with tracking.

AWS Equivalent: AWS Transfer Family

Floci Local Usage:

```
aws --endpoint-url=http://localhost:4566 transfer create-server --protocols SFTP --identity-provider-type SERVICE_MANAGED
```

.NET Developer: Legacy FTP client se aaya data → S3 mein automatic sync → .NET app process kare.

Project Example: Old cow farm software FTP se CSV export karta hai → Transfer Family se S3 mein → .NET app import kare.

◆ 51. Pricing / Cost Explorer / CUR / BCM Data Exports

Kya hai? AWS cost aur usage ko track, analyze, export karo.

Real-Life Example: Monthly budget planner — kahan kitna kharcha hua, kahan bachat ho sakti hai.

AWS Equivalent: AWS Cost Explorer, AWS Pricing API, AWS Billing and Cost Management

Floci Local Usage:

```
# Pricing API se service rates dekho
aws --endpoint-url=http://localhost:4566 pricing get-products --service-code AmazonS3

# Cost Explorer se usage analyze karo
aws --endpoint-url=http://localhost:4566 ce get-cost-and-usage --time-period
    Start=2024-01-01,End=2024-01-31 --granularity DAILY --metrics BlendedCost
```

.NET Developer:

```
var ce = new AmazonCostExplorerClient(creds, new AmazonCostExplorerConfig {
    ServiceURL="http://localhost:4566" });
var cost = await ce.GetCostAndUsageAsync(new GetCostAndUsageRequest {
    TimePeriod = new DateInterval { Start = "2024-01-01", End = "2024-01-31" },
    Granularity = Granularity.DAILY,
    Metrics = new List<string> { "BlendedCost" }
});
```

Project Example: Real AWS pe jaane se pehle cost estimate karo — Floci mein dummy data se practice.

Phase 1: Foundation (Week 1-2)

Services: S3, DynamoDB, SQS, SNS

Kya seekhna hai: - File storage ka basic (S3) - NoSQL database (DynamoDB) - Message queue system (SQS) - Pub-sub notifications (SNS)

.NET Practice: - ASP.NET Core API banaye - S3 mein file upload/download - DynamoDB mein CRUD operations - SQS mein message bhejo, worker se nikaalo - SNS se alert trigger karo

Mini Project: Cow Photo Upload System - Farmer photo upload kare → S3 mein store → SNS notification → DynamoDB mein metadata

Phase 2: Serverless & APIs (Week 3-4)

Services: Lambda, API Gateway, Cognito, Secrets Manager

Kya seekhna hai: - Bina server ke code chalao (Lambda) - REST APIs expose karo (API Gateway) - User login system (Cognito) - Secure password storage (Secrets Manager)

.NET Practice: - .NET Lambda function build karo (AWS Lambda templates) - API Gateway pe endpoint configure karo - Cognito User Pool mein farmer/admin banao - JWT token validate karo .NET API mein - Secrets Manager se DB password fetch karo

Mini Project: Secure Cow API - Login → JWT token → Cow data access → Lambda trigger on new entry

Phase 3: Infrastructure & Events (Week 5-6)

Services: RDS, CloudFormation, CloudWatch Logs, EventBridge

Kya seekhna hai: - SQL database setup (RDS/PostgreSQL via Docker) - Infrastructure as Code (CloudFormation) - Application logs (CloudWatch) - Event-driven architecture (EventBridge)

.NET Practice: - Entity Framework Core + PostgreSQL - YAML file se S3, DynamoDB, Lambda create karo (CloudFormation) - Serilog se CloudWatch mein logs bhejo - EventBridge rules se Lambda trigger karo

Mini Project: Complete Farm Backend - SQL tables for orders, customers - CloudFormation template for full setup - EventBridge + Lambda for business logic - CloudWatch mein debugging

Phase 4: DevOps & Containers (Week 7-8)

Services: ECR, ECS, CodeBuild, CodeDeploy, Step Functions

Kya seekhna hai: - Docker images store (ECR) - Container orchestration (ECS) - Automated build (CodeBuild) - Automated deploy (CodeDeploy) - Workflow automation (Step Functions)

.NET Practice: - Dockerfile se .NET app image banao - Docker Compose / ECS pe chhalao - buildspec.yml se CI pipeline banao - Step Functions se approval workflow banao

Mini Project: CI/CD Pipeline - Push → Build → Test → ECR Push → ECS Deploy → Step Functions approval

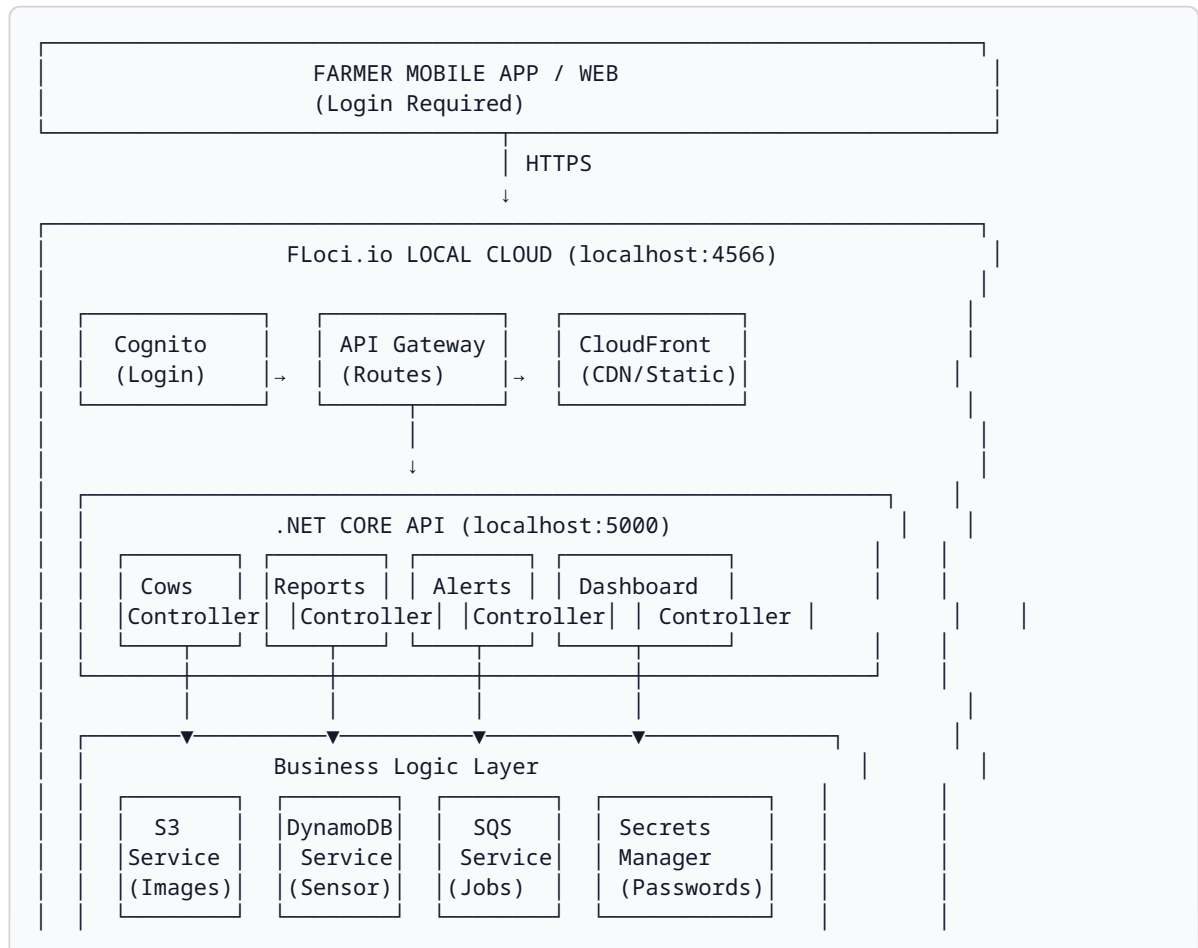


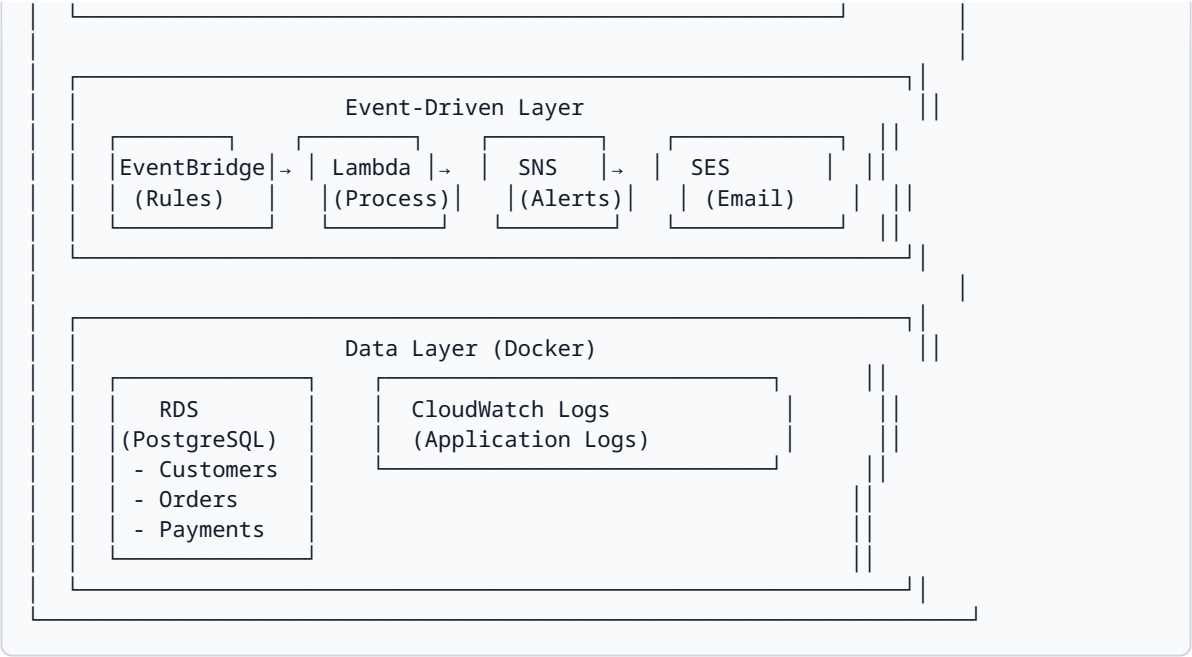
Complete Project: Smart Cow Health Monitoring Local Cloud System

Project Overview

Ek complete .NET backend jo local Floci pe chalta hai — cow health track karta hai, alerts bhejta hai, reports generate karta hai.

Architecture (Simple Hindi Text Diagram)





Service Usage Breakdown

Component	Floci Service	Kya Kaam Karta Hai?
Login	Cognito	Farmer/admin login, JWT token
Cow Images	S3	Cow photos, reports store
Sensor Data	DynamoDB	IoT temperature/humidity readings
Background Jobs	SQS	Heavy processing line mein lagao
Alerts	SNS	SMS/Email notifications
Business Logic	Lambda	Auto-process on events
APIs	API Gateway	Mobile app connect kare
SQL Data	RDS (PostgreSQL)	Customers, orders, payments
Secrets	Secrets Manager	DB password, API keys
Logs	CloudWatch Logs	Error tracking, debugging
Infrastructure	CloudFormation	Ek command se sab setup

Data Flow Example (Cow Fever Alert)



7. SQS → Background job for vet appointment scheduling
↓
8. CloudWatch → Log entry: "Alert sent at 14:30"
↓
9. RDS → Update cow status in relational database

.NET Project Structure

```
SmartCowLocal/
├── src/
│   ├── SmartCow.Api/                ← ASP.NET Core Web API
│   │   ├── Controllers/
│   │   │   ├── AuthController.cs    ← Cognito login
│   │   │   ├── CowsController.cs    ← CRUD + images
│   │   │   ├── SensorsController.cs ← Sensor data ingest
│   │   │   └── ReportsController.cs ← Report generation
│   │   ├── Services/
│   │   │   ├── S3Service.cs          ← S3 upload/download
│   │   │   ├── DynamoDbService.cs   ← NoSQL operations
│   │   │   ├── SqsService.cs        ← Queue operations
│   │   │   ├── SnsService.cs        ← Notifications
│   │   │   ├── RdsService.cs        ← SQL/EF Core
│   │   │   └── SecretsService.cs     ← Secret retrieval
│   │   ├── Models/
│   │   └── appsettings.json
│   ├── SmartCow.Lambda/             ← .NET Lambda functions
│   │   └── Function.cs              ← Fever alert processor
│   └── SmartCow.Shared/              ← Common models
├── infra/
│   └── cloudformation.yaml          ← Full infrastructure
├── docker-compose.yml              ← Floci + PostgreSQL
├── Dockerfile                      ← .NET API container
└── README.md
```

Key .NET Configuration (appsettings.Development.json)

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Host=localhost;Port=5432;Database=smartcow;Username=admin;Password=admin123"
  },
  "AWS": {
    "ServiceURL": "http://localhost:4566",
    "AccessKey": "test",
    "SecretKey": "test",
    "Region": "us-east-1"
  },
  "Cognito": {
    "UserPoolId": "us-east-1_localpool",
    "ClientId": "localclientid",
    "Authority": "http://localhost:4566/us-east-1_localpool"
  },
  "S3": {
    "BucketName": "smartcow-images"
  },
  "SQS": {
    "QueueUrl": "http://localhost:4566/000000000000/cow-jobs"
  },
  "SNS": {
    "TopicArn": "arn:aws:sns:us-east-1:000000000000:cow-alerts"
  }
}
```



```
}  
}
```

CloudFormation Template (infra/cloudformation.yaml)

```
AWSTemplateFormatVersion: '2010-09-09'  
Description: 'Smart Cow Local Infrastructure'
```

```
Resources:
```

```
  # S3 Bucket for Images
```

```
  CowImageBucket:
```

```
    Type: AWS::S3::Bucket
```

```
    Properties:
```

```
      BucketName: smartcow-images
```

```
  # DynamoDB Table for Sensor Readings
```

```
  SensorTable:
```

```
    Type: AWS::DynamoDB::Table
```

```
    Properties:
```

```
      TableName: sensor-readings
```

```
      BillingMode: PAY_PER_REQUEST
```

```
      AttributeDefinitions:
```

```
        - AttributeName: cowId
```

```
          AttributeType: S
```

```
        - AttributeName: timestamp
```

```
          AttributeType: S
```

```
      KeySchema:
```

```
        - AttributeName: cowId
```

```
          KeyType: HASH
```

```
        - AttributeName: timestamp
```

```
          KeyType: RANGE
```

```
      StreamSpecification:
```

```
        StreamViewType: NEW_IMAGE
```

```
  # SQS Queue for Background Jobs
```

```
  CowJobQueue:
```

```
    Type: AWS::SQS::Queue
```

```
    Properties:
```

```
      QueueName: cow-jobs
```

```
  # SNS Topic for Alerts
```

```
  CowAlertTopic:
```

```
    Type: AWS::SNS::Topic
```

```
    Properties:
```

```
      TopicName: cow-alerts
```

```
  # Secrets Manager for DB Password
```

```
  DbSecret:
```

```
    Type: AWS::SecretsManager::Secret
```

```
    Properties:
```

```
      Name: cow-db-password
```

```
      SecretString: '{"password":"admin123"}'
```

```
  # CloudWatch Log Group
```

```
  ApiLogGroup:
```

```
    Type: AWS::Logs::LogGroup
```

```
    Properties:
```

```
      LogGroupName: /smartcow/api
```

```
      RetentionInDays: 7
```

```
  # Lambda Execution Role
```

```
  LambdaRole:
```

```
    Type: AWS::IAM::Role
```

```
    Properties:
```

```
RoleName: cow-lambda-role
AssumeRolePolicyDocument:
  Version: '2012-10-17'
  Statement:
    - Effect: Allow
      Principal:
        Service: lambda.amazonaws.com
      Action: sts:AssumeRole
ManagedPolicyArns:
  - arn:aws:iam::aws:policy/AmazonDynamoDBFullAccess
  - arn:aws:iam::aws:policy/AmazonSNSFullAccess
```



Floci Se Real AWS Pe Migrate Karna

Migration Strategy

Step	Floci (Local)	Real AWS	Changes Needed
1	http://localhost:4566	https://s3.ap-south-1.amazonaws.com	ServiceURL change
2	test / test credentials	Real IAM User/Role credentials	Credentials swap
3	Docker Compose local	AWS Console / CLI / Terraform	Deployment method
4	us-east-1 default	ap-south-1 (Mumbai)	Region change
5	Free	Pay per use	Cost monitoring

Code Changes Required

```
// BEFORE (Floci Local)
var s3Config = new AmazonS3Config {
    ServiceURL = "http://localhost:4566",
    ForcePathStyle = true // Important for Floci!
};
var s3 = new AmazonS3Client(new BasicAWSCredentials("test","test"), s3Config);

// AFTER (Real AWS)
var s3Config = new AmazonS3Config {
    RegionEndpoint = RegionEndpoint.APSouth1 // Mumbai
    // ForcePathStyle = false (default, works with real AWS)
};
var s3 = new AmazonS3Client(new EnvironmentVariablesAWSCredentials(), s3Config);
// Real AWS mein credentials from ~/.aws/credentials ya IAM Role
```

Configuration Changes

```
// BEFORE (Floci)
{
  "AWS": {
    "ServiceURL": "http://localhost:4566",
    "AccessKey": "test",
```

```

    "SecretKey": "test",
    "Region": "us-east-1"
  }
}

// AFTER (Real AWS)
{
  "AWS": {
    "Region": "ap-south-1"
    // ServiceURL removed – default AWS endpoint use hoga
    // AccessKey/SecretKey removed – IAM Role / Environment credentials
  }
}

```

Deployment Migration Steps

```

# Step 1: Floci mein test karo
aws --endpoint-url=http://localhost:4566 cloudformation create-stack --stack-name
cow --template-body file://infra.yaml

# Step 2: Same template real AWS pe (bas endpoint hatao)
aws cloudformation create-stack --stack-name cow --template-body file://infra.yaml --
capabilities CAPABILITY_IAM

# Step 3: .NET app deploy karo
# ECS/Fargate pe:
aws ecr get-login-password | docker login --username AWS --password-stdin
<account>.dkr.ecr.ap-south-1.amazonaws.com
docker build -t cow-api .
docker tag cow-api:latest <account>.dkr.ecr.ap-south-1.amazonaws.com/cow-api:latest
docker push <account>.dkr.ecr.ap-south-1.amazonaws.com/cow-api:latest

# Step 4: Update ECS service
aws ecs update-service --cluster cow-cluster --service cow-api-service --force-new-
deployment

```



Summary (Ek Line Mein)

“Floci pe seekho, practice karo, debug karo — jab .NET code aur CloudFormation template perfect ho, tab sirf endpoint URL badlo aur real AWS pe deploy karo! Koi architectural change nahi, koi code rewrite nahi — sirf localhost:4566 se AWS Mumbai (ap-south-1) pe switch!”

Cheat Sheet: 51 Services ka Quick Reference

#	Service	Hindi mein Kaam	.NET mein Kahan Use?
1-4	S3, SQS, SNS, SES	Storage, Queue, Alerts, Email	FileService, NotificationService
5-6	DynamoDB, Streams	NoSQL DB, Change events	SensorRepository, EventHandlers

#	Service	Hindi mein Kaam	.NET mein Kahan Use?
7	Lambda	Serverless code	Background processors
8-9	API Gateway v1/ v2	REST/WebSocket APIs	API exposure, real-time
10	Cognito	Login/Auth	JWT middleware, AuthController
11-12	IAM, STS	Permissions, Temp access	Role management
13-16	KMS, Secrets, SSM, ACM	Encryption, Secrets, Config, SSL	SecurityService
17-22	EventBridge, Pipes, Scheduler, Step Functions, Kinesis, Firehose	Events, Workflows, Streams	Event-driven architecture
23-26	RDS, ElastiCache, MSK, OpenSearch	SQL, Cache, Kafka, Search	EF Core, Redis, Elasticsearch
27-32	ECR, ECS, EKS, EC2, Auto Scaling, ELB	Containers, VMs, Scale	DevOps, Docker, Kubernetes
33-38	CloudFormation, CodeBuild, CodeDeploy, AppConfig, Tagging	Infrastructure, CI/CD	DevOps pipeline
39-42	CloudWatch Logs, Metrics, Config, Backup	Monitoring, Compliance	Serilog, Dashboards
43-47	Glue, Athena, Bedrock, Textract, Transcribe	Data, AI, Documents	Analytics, OCR, AI
48-51	CloudFront, Route53, Transfer, Pricing	CDN, DNS, SFTP, Billing	Global delivery

Happy Cloud Learning! Floci pe practice karo, real AWS pe confidence se deploy karo! 🚀☁️🇮🇳